Fuzzy ODEs as a Means for Low-Thrust Trajectory Generation

Nathan Steffen, Magnus Sieverding, and Kelly Cohen

Abstract Optimal low-thrust trajectory optimization is computationally expensive due to long transfer durations and complex dynamics, motivating the use of sub-optimal trajectories for warm-starting optimization. While recent machine learning approaches enable efficient trajectory generation, they often rely on simplified dynamics, lack robustness to problem formulation, and operate as non-auditable black boxes—limitations that are undesirable for mission-critical guidance. This work proposes a fuzzy logic–based approach inspired by neural Ordinary Differential Equations (ODEs). Fuzzy ODEs combine computational efficiency with interpretability and robustness, making them well suited for trajectory generation. A Fuzzy ODE architecture is developed and evaluated on a Geosynchronous Transfer Orbit to Geosynchronous Orbit problem with fuel minimization as the objective. Results demonstrate the method's ability to leverage higher-fidelity dynamics to effectively target the semi-major axis, while highlighting opportunities to extend the approach for improved control of additional state variables in future work.

1 Introduction

Electric propulsion (EP) was first proposed in the early 20th century and practically realized in the 1960s [1]. Offering significantly higher propellant efficiency at the cost of lower thrust, EP has increasingly replaced high-thrust chemical propulsion for many missions, including Dawn [2], Deep Space 1 [3], and the Lunar Gateway Power and Propulsion Element [4]. However, the low-thrust and continuous-operation nature of EP results in long transfer durations and more complex dynamical behavior, making the computation of optimal low-thrust trajectories a persistent and computationally demanding challenge in mission design.

Nathan Steffen, Magnus Sieverding, Kelly Cohen
University of Cincinnati, 2600 Clifton Ave, Cincinnati OH 45221, USA
e-mail: steffenr@mail.uc.edu, sievermf@mail.uc.edu, cohenky@ucmail.uc.edu

The two ways to optimize a low-thrust orbit transfer are the direct and indirect methods. Indirect methods utilize first-order necessary conditions to determine optimality with the help of co-states. In contrast, direct methods rely on transcription techniques to approximate the dynamics and controls of the system thus reformulating the optimal controls problem as a nonlinear program [5]. While direct methods are the standard for most modern spacecraft mission optimizations, both optimization methods are reliant on initial guesses, so the ability to find an optimal or feasible solution depends on the trajectory initially given, or the amount of time that one is willing to spend on the optimization.

It is desired that there are efficient and reliable ways to warm start a low-thrust orbit transfer optimization for a few reasons. Primarily, it aids the mission designer in finding solutions for a successful mission, thus enabling more optimal missions to be designed overall. Additionally, since the EP engine is being utilized continuously, there is a non-trivial probability that the engine may break down over the course of an orbit transfer. This is known as a missed thrust event, and in this case, being able to generate an efficient solution could be the difference between mission success and failure. If the engine is out for long enough, a trajectory will need to be generated to bring the crew and/or payload to safety should the original mission become unobtainable.

Generating suboptimal trajectories for warm-starting an optimization has been a popular means to get around initial guess issues inherent to direct and indirect methods [6, 7, 8, 9]. While analytical methods do exist for low-thrust trajectory warm starting [10, 11], many cutting-edge approaches utilize machine learning (ML) [5, 6, 12, 13, 14, 15]. These ML approaches have proven to be capable for the specific problems they address, but their current application is limited and brittle. Typically, models are trained extensively on a singular set of initial conditions over a multitude of feasible and optimal trajectories. This means that if these initial conditions are perturbed even by a small amount, the ML model will not be able to perform as well, if at all. Additionally, these ML models are limited to assumptions in their dynamic systems, such as pure two-body effects, or the circularly restricted three body problem. In a real low-thrust orbit transfer, external forces due to the zonal harmonics of Earth's gravitational field or the gravitational pull from other bodies like the Moon are present and have a non-negligible influence on the trajectory.

Fuzzy Ordinary Differential Equations (FODEs) offer a viable alternative to address these gaps. Most recently utilized by Güven and Kumbasar [16], FODEs are Fuzzy Inference Systems (FISs) that learn the underlying physics of a system, rather than learning patterns in data. This approach is inspired by the success of Neural ODEs (NODEs) [17], which instead uses deep neural networks (DNNs) to learn the physics of a dynamic system. Regardless of the backbone, this approach offers a more robust means to trajectory generation. While it works well for pure simulation data, it was shown in Sandoval et al.'s work [18] that NODEs can be used as optimal policies in a dynamic system, similar to Reinforcement Learning (RL). The key difference between the two methods is in how they are trained. RL relies on feature space sampling, which can be inefficient for training – especially in a large

feature space like low-thrust trajectories have. NODEs and FODEs are trained by integrating through full trajectory and then back-propagating to train according to some cost function.

Now, instead of just learning the physics of a system, the optimal controls at any given state may also be learned. Once higher fidelity effects are included as channels, this approach has the potential to robustly learn how to efficiently produce a physical, near-optimal trajectory for a low-thrust orbit transfer problem so that a direct optimization can be well warm started. The goal of this work is to develop a Fuzzy ODE approach for optimal policy learning accounting for more complicated dynamic effects, and to evaluate its performance against a state-of-the-art analytical method.

2 Related Work

The state of the art for low-thrust trajectory generation can be split into two categories: analytic methods and machine learning methods.

Analytical methods for low-thrust trajectory design generate suboptimal transfers using closed-form or heuristic formulations rather than learned models. Common approaches include metric-based guidance laws such as Q-Law and DAG, which define a scalar distance-to-target measure and derive thrust directions accordingly [10, 11, 19]. While these methods are broadly applicable and physics-grounded, they can be computationally expensive and yield trajectories that approach the target orbit asymptotically rather than terminating exactly at it. In practice, this limitation is typically addressed via thresholding and a terminal optimization stage. Lambert's Problem-based methods provide complete boundary-value solutions but are generally prohibitively expensive for low-thrust problems and are therefore less practical for repeated evaluations [20].

Machine learning has been explored as a means of mitigating computational cost and producing complete trajectories. Diffusion models have recently been applied to learn distributions of low-thrust transfers and generate full trajectories for warm-starting optimization [5], but these approaches are data-intensive and lack explicit physical structure, making them sensitive to deviations in initial conditions. Reinforcement learning has also been applied [21], though the large state–action space leads to sampling inefficiencies and limited robustness. Behavior cloning offers an alternative by learning control histories from optimal solutions [14]; however, the absence of embedded physics necessitates corrective steps such as Neural ODE refinement and still omits important perturbative effects, including third-body gravity and non-uniform Earth gravitational effects.

3 Preliminaries

3.1 Orbital Elements

An orbit is described by six parameters known as the classical (Keplerian) orbital elements, which provide an intuitive and widely used framework for orbit definition and visualization [20]. These elements specify the orbit's size, shape, orientation, and the spacecraft's location along the trajectory.

The semi-major axis (SMA) defines the orbit's size, while the eccentricity (ECC) characterizes its shape; for low-thrust transfers, ECC is typically constrained to $[0,1)$ to ensure a closed orbit and avoid numerical issues. Inclination (INC) specifies the orbit's tilt relative to the equatorial plane, and the remaining angular elements—right ascension of the ascending node (Ω), argument of periapsis (ω), and true anomaly (TA)—define the orbit's orientation and the spacecraft's instantaneous position along it. Figure 1 can be used as a reference for visualizing these orbital elements.

While classical orbital elements (COEs) are well suited for visualization and problem formulation, they are ill-conditioned for simulation because several elements become undefined for circular ($e = 0$), equatorial ($i = 0°$), or polar ($i = 90°$) orbits. To address these singularities, alternative orbital element sets have been developed that are numerically robust across a broader range of orbital regimes.

In this work, trajectories are specified using COEs but propagated using equinoctial elements (EEs). Although less intuitive, EEs remain well defined for circular and equatorial orbits (polar orbits are not considered here). The semi-major axis is retained, while eccentricity and orientation are represented by the eccentricity vector components (e_x, e_y) and node vector components (n_x, n_y). The final element is the true longitude L, which specifies the spacecraft's position along the orbit and serves a role analogous to the true anomaly.

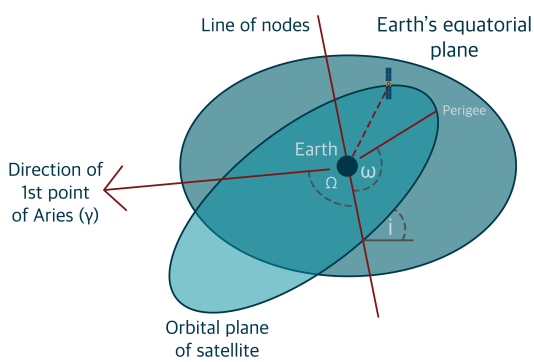

Fig. 1 Description of an orbit with Keplerian orbital elements, from [22].

3.2 Gauss' Variational Equations

Gauss' Variational Equations (GVEs) are a set of ordinary differential equations that dictate the rate of change of the orbital elements given perturbations in the radial, tangential, and normal directions [20]. These radial, tangential, and normal directions describe the frame that the spacecraft experiences, where the radial direction points to the center of the Earth, the tangential direction points along the velocity vector, and the normal direction completes the cross product of the radial and tangential vectors.

The formulation for GVEs is seen in Equation 1 [19]. This is a simplified matrix form since the actual equations are quite long, but a full description of these equations can be seen in [19].

$$\begin{bmatrix} \dot{a} \\ \dot{e}_x \\ \dot{e}_y \\ \dot{n}_x \\ \dot{n}_y \\ \dot{L} \end{bmatrix} = \begin{bmatrix} C_{ar} & C_{at} & 0 \\ C_{e_x r} & C_{e_x t} & C_{e_x n} \\ C_{e_y r} & C_{e_y t} & C_{e_y n} \\ 0 & 0 & C_{n_x n} \\ 0 & 0 & C_{n_y n} \\ 0 & 0 & C_{Ln} \end{bmatrix} \begin{bmatrix} \Delta_r \\ \Delta_t \\ \Delta_n \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ C_L \end{bmatrix} \tag{1}$$

In Equation (1), Δ_r, Δ_t, and Δ_n are the radial, tangential, and normal perturbations that the spacecraft experiences. Let $\boldsymbol{\Delta} = [\Delta_r \ \Delta_t \ \Delta_n]^T$ be the perturbing vector. Then $\boldsymbol{\Delta} = \boldsymbol{\Delta}_{\text{moon}} + \boldsymbol{\Delta}_{\text{zonal harmonics}} + \boldsymbol{\Delta}_{\text{thrust}}$ for this use case. For $\boldsymbol{\Delta}_{\text{thrust}} = [F_r \ F_t \ F_n]^T$, Equations (2)-(4) describe how the spacecraft's thrust perturbs the orbit.

$$F_r = F \cos\beta \sin\alpha \tag{2}$$

$$F_t = F \cos\beta \sin\alpha \tag{3}$$

$$F_n = F \cos\beta \sin\alpha \tag{4}$$

Here, F is the thrust magnitude applied by the spacecraft, β is the out-of-plane thrust direction, and α is the in-plane thrust direction. α primarily influences the smi-major axis, eccentricity, and true anomaly, while β primarily influences the inclination and right ascension of the ascending node; both angles jointly affect the argument of periapsis. By steering the thrust vector $\boldsymbol{\Delta}_{\text{thrust}}$ to exploit natural perturbations that already drive favorable changes in specific orbital elements, the spacecraft can reduce fuel expenditure. This principle motivates the inclusion of perturbation effects as explicit input channels to the FODE model introduced later.

The mass is also considered a state of the system, and its time derivative is specified by $\dot{m} = -T/(mI_{sp}g_0)$. Here, I_{sp} is known as the specific impulse and it is a measure of the engine's efficiency. A higher specific impulse means a lower mass rate of change per timestep. Additionally, g_0 is a constant known as standard gravity, and its value is 9.80665 m/s^2.

4 Methodology

A Fuzzy ODE is a FIS that learns the physics that produces a trajectory, rather than learning the patterns seen in a trajectory. In other words, if $\dot{\mathbf{x}} = A\mathbf{x} + B\mathbf{u}$ is the true dynamics of a system, where $\mathbf{x}$ is the state vector, $\mathbf{u}$ is the control vector, and $\dot{\mathbf{x}}$ is the state rate of change vector, then the FODE learns to mimic $\dot{\mathbf{x}}$.

In Sandoval et al.'s work[18], the authors note that a NODE only needs to have the form $\dot{\mathbf{x}} = f(\mathbf{x}(t), \theta)$, and that the function itself could already utilize information from the dynamics. In this case, the learnable function may instead look like $\dot{\mathbf{x}} = A\mathbf{x} + Bf(\mathbf{x}, \theta)$, where now only the control vector is being learned. This results in a novel means to train control policies, where traditionally RL has been the go-to method. In this work, a FODE is used in place of an NODE.

The proposed methodology takes inspiration from Sandoval et al. [18], but differs slightly from what the authors propose. There is still a two step process to training this FODE, namely preconditioning (Section 4.3) and policy training (Section 4.4). Before describing those methods, it is worth discussing the specific problem this model aims to address (Section 4.1), along with the model itself (Section 4.2).

4.1 Problem Statement

The problem that this model is trained on is the geosynchronous transfer orbit (GTO) to geosynchronous orbit (GEO) problem, otherwise known as GTO to GEO. GTO has bounds on its initial orbit, as seen in Table 1, where a given case to train on is uniformly randomly selected. Each LEO case has the same GEO end point, however, where the target SMA is 42,000 km, the target ECC is 0.001, and the target INC is 0.05^{o} – a near circular equatorial orbit. The other target parameters are not considered for this problem. The initial mass of the vehicle is 2,000 kg, the thrust is 0.35 N, and the specific impulse is 2,000 s [23].

Table 1 GTO Initial Condition Bounds.

Variable	Lower Bound	Upper Bound
Reference Date	January 1 2026	January 1 2027
SMA	22,055.3 km	26,956.5 km
ECC	0.6	0.8
INC	0.5^{o}	15°
RAAN	0°	360°
AOP	-180°	180°
TA	0°	360°

4.2 Model

The model developed in this work is called GTO2GEOFIS for its use in this specific orbit transfer, and a diagram visualizing it can be seen in Figure 2. This is a three-layered cascading FIS where the inputs to the first layer are the scaled EEs. These are divided into the RT and N FISs. RT encodes the radial and tangential direction two-body importance since its input variables are those that are most changed by radial and tangential perturbations, and N encodes the normal direction two-body importance for similar reasons. These values get passed to the second layer where external perturbations are introduced.

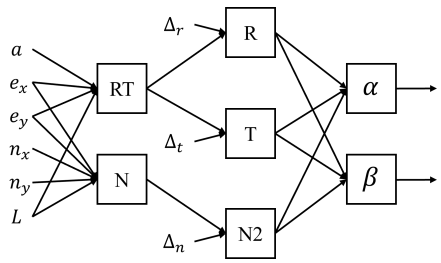

Fig. 2 GTO2GEOFIS.

Every FIS present in GTO2GEOFIS relies on trapezoidal membership functions with Ruspini partitioning. The FISs in the first layer of GTO2GEOFIS both contain only 3 membership functions per input so that the rule base is kept manageable, along with order 0 Takagi-Sugeno-Kang (TSK) rules. The second layer contains 7 membership functions per input since there are less inputs present with order 1 TSK, and finally the third layer contains 11 membership functions with order 2 TSK as an attempt to ensure smoother output behavior.

4.3 Behavior Cloning

To precondition GTO2GEOFIS, behavior cloning is utilized on a dataset generated by the heuristic-based method known as Q-Law [23]. This differs from what was originally proposed in [18] since the authors intended for preconditioning to be performed in simulation over increasing time windows. Instead, GTO2GEOFIS was trained as a simple regression problem on the trajectories generated from Q-Law, similar to what is done in behavior cloning [14]. Here, mean squared error is used as the loss function.

With the bounds provided in Table 1, 100,000 trajectories were generated using Q-Law, and this became the preconditioning training set. These trajectories were then split 70-15-15 for training, validation, and testing. Overall, there were around

64 million data points utilized in this split. For more information on how Q-Law functions, please refer to Steffen et al. [11].

4.4 Policy Training

Once the model is preconditioned, it is ready to propagate a simulation and be fine-tuned. The cost function used for this procedure can be seen in Equation (5). The first two terms for this cost function punish divergence from the desired solution (GEO orbit), and the last term punishes fuel expenditure. Since it is assumed that the engine does not turn off throughout this trajectory, the minimum fuel use solution is the same as the minimum time of flight solution, so no specification needs to be made to balance the two solutions.

$$\mathscr{C} = \left(\frac{SMA_f - 42,000}{42,000}\right)^2 + (ECC_f - 0.001)^2 + (INC_f - 0.05)^2 - \frac{m_f}{2,000} \quad (5)$$

GTO2GEOFIS and the dynamics it propagates are implemented in JAX[24] using Equinox and Diffrax [25]. These packages not only enable gradient backpropagation through a trajectory for optimization, but they also speed up training using just-in-time compilation, vectorization mapping, and GPU acceleration, making them crucial to the efficiency in the training of this model, which is performed in a gradient-based manner similar to Sieverding et al. [26].

5 Results & Discussion

5.1 Behavior Cloning

Regression optimization for preconditioning was performed using the Adam optimizer with a learning rate of 0.001 and a batch size of 16,384. Mean squared error (MSE) was the loss function for this training, and the R^2 on the validation training set was tracked as well. For the sake of space, the training history plot is omitted, but the final testing dataset MSE was 0.0055 and the final testing R^2 value was 0.899, indicating a solid fit to the Q-Law-based dataset with room for improvement. 300,000 epochs were used for GTO2GEOFIS to clone Q-Law's behavior where GPU acceleration on an NVIDIA RTX 5070 Ti enabled the fast evaluation of this dataset.

5.2 Policy Training

Figure 3 shows the policy training history. In this training, GTO2GEOFIS is also trained with the Adam optimizer, but this time at a learning rate of 0.0001 with a batch size of 64. There were 10,000 different cases generated for GTO2GEOFIS to propagate with a 70-15-15 split for validation and testing. A single epoch with these hyperparameters took around 6 minutes or so for propagation of all 64 training cases, all 64 validation cases, and the computation of the gradients across the trajectory on an Intel Core Ultra 7 CPU. Issues were encountered for the utilization of GPU in this mode, so training had to fall back to CPU instead.

The training seen in Figure 3 is pretty noisy, indicating difficulty in training. This could also be due to the lack of consistency with the training cases used at each epoch, or it could be due to ill-fitted terminating conditions for GTO2GEOFIS. Regardless, increasing the effectiveness of this part of FODE training for optimal feedback control policies is certainly worth investigating further.

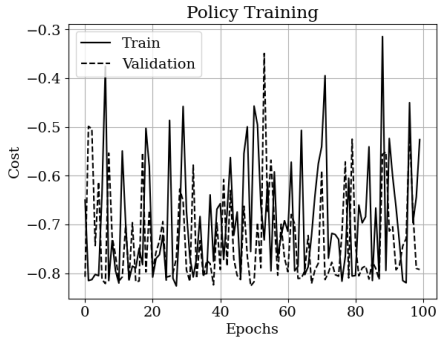

Fig. 3 Policy Training History

5.3 Q-Law Comparison

Figure 4 shows the time histories of the trained GTO2GEOFIS compared to the analytic method Q-Law. It can be seen that both the SMA and ECC appear to approach the respective target values, but INC diverges for GTO2GEOFIS. This is due to an error in the implemented cost function, and fixing this error is a goal for future work. Regardless, it can be seen in the SMA plot that GTO2GEOFIS is capable of increasing its SMA at a higher rate than that of Q-Law, indicating some learned behavior to utilize external perturbations for increased benefit.

The control history of the two approaches can be seen in Figure 5. Most notably, the main consideration here is the behavior of the out-of-plane thrust angles, where

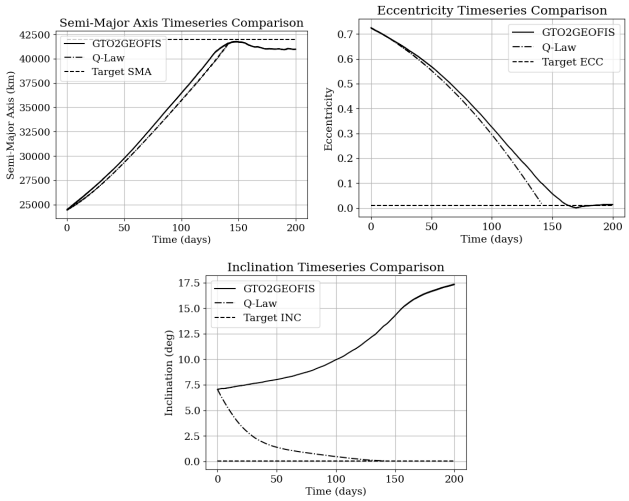

Fig. 4 Relevant state history of GTO2GEOFIS and Q-Law.

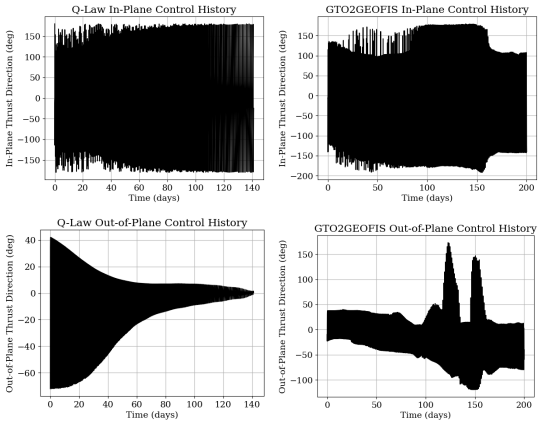

Fig. 5 Control history of GTO2GEOFIS and Q-Law.

it can be seen that there is no decaying behavior for GTO2GEOFIS like there is with Q-Law, further showcasing how it did not learn the out-of-plane thrust angle correctly. Finally, Figure 6 shows a visualization of the orbit transfer in the XY, XZ, and YZ planes. Most notably, it can be seen that the tilt of the orbit (inclination) increases for GTO2GEOFIS rather than decreases like it should with Q-Law.

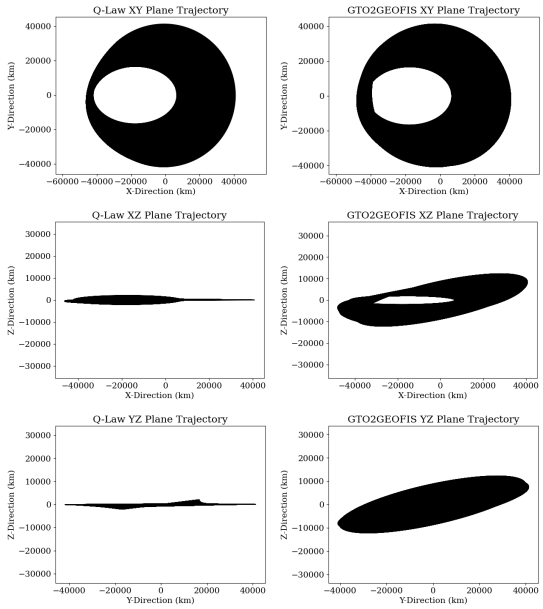

Fig. 6 Orbit Transfer Visualization.

6 Conclusion

This paper showcased a novel way for generating a low-thrust trajectory using a cascading FIS that learns the dynamics of a system, rather than just the patterns present in data – otherwise known as a Fuzzy ODE. The model development was shown, and its 2 step training was demonstrated as well. At the end of training, it was compared to a state-of-the-art analytical method, where it was found that mistakes were made with the original training of this FODE resulting in unexpected behavior, at least for inclination targeting. However, perturbation exploitation behavior was seen in semi-major axis targetting, indicating use in this model's further development. This approach shows potential in its efficiency for training a FIS for physical systems in a gradient-based manner for applications beyond low-thrust orbit transfers.

In the future, more low-thrust problems should be tested with this approach, and further development of the GTO2GEOFIS system should be investigated to provide more accurate and robust results.

References

1. Choueiri, E.Y.: A critical history of electric propulsion: the first 50 years. *Journal of Propulsion and Power* **20**(2), 193–203 (2004)

2. NASA Science, "Dawn," NASA Science Mission Pages, https://science.nasa.gov/mission/dawn/ (accessed Feb. 5, 2026).
3. NASA Science, "Deep Space 1," NASA Science Mission Pages, https://science.nasa.gov/mission/deep-space-1/ (accessed Feb. 5, 2026).
4. NASA, "A Powerhouse in Deep Space: Gateway's Power and Propulsion Element," NASA Missions – Artemis/Gateway, https://www.nasa.gov/missions/artemis/gateway/a-powerhouse-in-deep-space-gateways-power-and-propulsion-element/ (accessed Feb. 5, 2026).
5. Gräbner, J., Beeson, R.: Global search for optimal low thrust spacecraft trajectories using diffusion models and the indirect method. arXiv preprint arXiv:2501.07005 (2025). Accepted for presentation at the 35th AAS/AIAA Space Flight Mechanics Meeting.
6. Shi, J., Wang, J., Su, L., Ma, Z., Chen, H.: A neural network warm-started indirect trajectory optimization method. *Aerospace* **9**(8), 435 (2022)
7. Ren, H., Niu, Y., Li, Y., Yang, L., Gao, H.: Automatic parking trajectory planning based on warm start nonlinear dynamic optimization. *Sensors* **25**(1), 112 (2025)
8. Cheng, Z.: Learning warmstart: accelerating trajectory optimization with self-attention Siamese networks. Master's thesis, Worcester Polytechnic Institute (2025)
9. Guffanti, T., Gammelli, D., D'Amico, S., Pavone, M.: Transformers for trajectory optimization with application to spacecraft rendezvous. In: *IEEE Aerospace Conference* (2024)
10. Petropoulos, A.E.: Simple control laws for low-thrust orbit transfers. Acta Astronautica **133**(1), 49–60 (2017)
11. Steffen, N., Falck, R., Faller, B.: Q-law for rapid assessment of low-thrust cislunar trajectories via automatic differentiation. In: *Proceedings of the AAS/AIAA Space Flight Mechanics Meeting*, 2025.
12. Zhang, Z., Topputo, F.: Neural approximators for low-thrust trajectory transfer cost and reachability. arXiv preprint arXiv:2508.02911 (2025)
13. Izzo, D., Öztürk, E.: Real-time optimal guidance and control for interplanetary transfers using deep networks. arXiv preprint arXiv:2002.09063 (2020)
14. Origer, S., Izzo, D.: Closing the gap: optimizing guidance and control networks through neural ODEs. arXiv preprint arXiv:2404.16908 (2024)
15. Guo, X., Ren, D., Wu, D., Jiang, F.: DNN estimation of low-thrust transfer time for multi-asteroid rendezvous missions. *Acta Astronautica* **204**, 518–530 (2023)
16. Güven, Y., Kumbasar, T.: Fuzzy Logic Strikes Back: Fuzzy ODEs for dynamic modeling and uncertainty quantification. *IEEE Transactions on Artificial Intelligence*, pp. 1–9 (2025). doi:10.1109/TAI.2025.3557765
17. Chen, R.T.Q., Rubanova, Y., Bettencourt, J., Duvenaud, D.: Neural ordinary differential equations. In: *Advances in Neural Information Processing Systems* (NeurIPS) (2018)
18. Sandoval, I.O., Petsagkourakis, P., del Rio-Chanona, E.A.: Neural ODEs as feedback policies for nonlinear optimal control. arXiv preprint arXiv:2210.11245 (2022)
19. Sánchez Pérez, J.M., Varga, G.I.: Many-revolution low-thrust orbit transfer computation using equinoctial Q-law including J2 and eclipse effects. In: *AIAA/AAS Astrodynamics Specialist Conference* (2015)
20. Vallado, D.A.: *Fundamentals of Astrodynamics and Applications*. 5th edn. Microcosm Press, El Segundo (2022)
21. Syed, A., Arustei, A., Munir, A., Dutta, A.: Automated trajectory planning: a cascaded deep reinforcement learning approach for low-thrust spacecraft orbit-raising. *IEEE Aerospace and Electronic Systems Magazine* **14**(8), 1–10 (2025)
22. University of Southern Queensland: The position of satellites. In: *GPS and GNSS*. USQ Press (2021)
23. Petropoulos, A.: Low-thrust orbit transfers using candidate Lyapunov functions with a mechanism for coasting. In: *AIAA Guidance, Navigation, and Control Conference* (2003)
24. Bradbury, J. *et al.*: JAX: composable transformations of Python+NumPy programs. Software repository, GitHub (2018).
25. Kidger, P.: On neural differential equations. *Journal of Machine Learning Research* **23**(241), 1–104 (2022)
26. Sieverding, M., Steffen, N., Cohen, K.: Gradient-Optimized Fuzzy Classifier: a benchmark study against state-of-the-art models. arXiv preprint arXiv:2504.16263v1 (2025)